

Приватний заклад вищої освіти
Одеський технологічний університет «ШАГ»
Кафедра інформаційних технологій та фундаментальної підготовки

випускна кваліфікаційна робота бакалавра

**Інформаційна платформа розповсюдження мультимедійного контенту за
моделлю підписки (на прикладі сервісу Bazinga)**

на здобуття бакалаврського рівня вищої освіти
зі спеціальності «122 Комп'ютерні науки»

Виконавці проекту:

№	П.І.Б.	Ролі	Група
1	[Прізвище Ім'я По батькові автора 1]	Full-Stack	КН-П-XXX
2	[Прізвище Ім'я По батькові автора 2]	Full-Stack	КН-П-XXX

Автор звіту: [Прізвище Ім'я По батькові автора 2]

Керів- ник	Доцент кафедри ІТ та ФП, к.т.н., [Прізвище І. П. керівни- ка]
---------------	--

Одеса – 2025

АНОТАЦІЯ

[Прізвище І. П. автора 2]. Інформаційна платформа розповсюдження мультимедійного контенту за моделлю підписки (на прикладі сервісу Bazinga). Підсистема каталогу контенту, агрегатор метаданих та потокове відео. — випускна кваліфікаційна робота на здобуття бакалаврського рівня вищої освіти зі спеціальності «122 Комп'ютерні науки». — Одеський технологічний університет «ШАГ», Одеса, 2025.

Робота присвячена проєктуванню та повноцінній реалізації підсистеми каталогу контенту, агрегатора метаданих та потокового відео для розподіленої інформаційної платформи Bazinga, що поєднує у собі сервіс для читання коміксів та манги (Bazinga Comics) і сервіс потокового перегляду відеоконтенту (BazingaTV). Виконавцем як Full-Stack розробником у межах роботи здійснено як проєктування серверної частини відповідних модулів (на основі ASP.NET Core 9 та Entity Framework Core поверх MySQL), так і реалізацію клієнтської частини (на основі React 18, TypeScript та Vite з використанням бібліотеки компонентів shadcn/ui над Radix UI).

Актуальність роботи визначається стрімким зростанням обсягів цифрового контенту, доступного користувачам сервісів медіадистрибуції. Сучасні платформи дистрибуції коміксів і потокового відео налічують десятки тисяч одиниць контенту, що робить ключовою задачею не стільки зберігання, скільки відкриття: ефективні механізми пошуку, фільтрації та персоналізованої рекомендації. Окремим викликом є агрегація метаданих з зовнішніх джерел — далеко не всі сервіси можуть собі дозволити власну редакційну команду, що описує тисячі тайтлів вручну. Натомість сучасний підхід передбачає інтеграцію з відкритими або платними API-провайдерами метаданих, які постачають заздалегідь структуровані дані з обкладинками, описами, рейтингами та переліками акторів/творців.

Об'єктом дослідження виступає підсистема відображення, пошуку, фільтрації та потокового відтворення мультимедійного контенту в межах багаторівневої вебсистеми, побудованої за архітектурним стилем SPA + REST API. Предметом дослідження є методи проксіювання та кешування запитів до зовнішніх метаджерел, моделі компонентної архітектури для відображення великих списків контенту з пагінацією, а також підходи до інтеграції відеоплеєра в режимі повного екрана.

У ході роботи виконано такі задачі:

- проаналізовано доступні відкриті API-провайдери метаданих (Jikan для аніме/манги через MyAnimeList, akabab/superhero-api для персонажів коміксів, TVMaze для серіалів) та обрано їх як основні джерела даних для відповідних розділів платформи;

- спроектовано та реалізовано серверний агрегатор метаданих: контролер `MetadataController` з ендпоінтами для аніме, манги, персонажів і шоу; сервіси `JikanService` і `SuperheroService` з підтримкою кешування у пам'яті засобами `IMemoryCache`;
- реалізовано серверну частину для роботи з власною базою коміксів: контролери `ComicsController`, `CategoriesController`, `ConditionsController`, `CartController`, `WishlistController`, `LibraryController`, `NewsController`, а також їх адміністративні аналоги;
- реалізовано сервіс ціноутворення `PricingService`, що враховує тип підписки користувача при розрахунку вартості одиниці контенту;
- реалізовано клієнтську частину: домашню сторінку `Bazinga Comics`, сторінку каталогу всіх коміксів, сторінку каталогу всієї манги, сторінку всіх персонажів, сторінку `BazingaTV` з каруселлю трейлерів, сторінку повноекранного перегляду відео, юридичні сторінки (About, FAQs, Careers, Internships);
- реалізовано уніфікований компонент картки контенту `MediaCard`, що використовується як для коміксів, так і для манги, забезпечуючи візуальну єдність всіх рядів у каталозі;
- реалізовано центральний пошуковий компонент `MasterSearch`, що працює одночасно за коміксами (локальний каталог), персонажами (`akabab`) і мангою (`Jikan`); відображається як накладений шар поверх будь-якої сторінки за натисканням `Cmd/Ctrl+K`;
- реалізовано модальні вікна детального перегляду для коміксу, манги і персонажа з підтримкою відображення з пошукового шару, рядів домашньої сторінки і каталогів;
- реалізовано анімовані каруселі (`HeroCarousel`, `TVHero`) з підтримкою автоматичного перемикавання, ручної навігації стрілками та індикаторами-крапками;
- здійснено інтеграційне тестування у браузерх `Chrome`, `Firefox` та `Edge`.

Результатом виконаної роботи є завершена, відмовостійка та підтримувана підсистема каталогу контенту та потокового відео, що задовольняє вимоги технічного завдання та готова до подальшої експлуатації у складі вебсистеми `Bazinga`.

ЗМІСТ

- ЗМІСТ
 - ВСТУП
 - ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ
 - РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ
 - РОЗДІЛ 2. РЕАЛІЗАЦІЯ ПІДСИСТЕМИ КАТАЛОГУ ТА АГРЕГАТОРА МЕТАДАНИХ
 - РОЗДІЛ 3. РЕАЛІЗАЦІЯ ПІДСИСТЕМИ ПОТОКОВОГО ВІДЕО ТА ПОШУКУ
 - ВИСНОВКИ
 - ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ
-

ВСТУП

Сучасні сервіси цифрової дистрибуції медіаконтенту вирізняються гранично великими каталогами: тільки на одній платформі може налічуватися десятки тисяч одиниць контенту, і ця кількість зростає щодня. Через таку щільність каталогу для користувача втрачає сенс «перегляд усього доступного»; натомість на перший план виходять інструменти ефективного відкриття контенту: тематичні рекомендації, гранульована фільтрація, повнотекстовий пошук, а також персоналізовані добірки на основі історії перегляду.

Окремим викликом для розробників подібних платформ є наповнення каталогу метаданими. Якісний опис тайтлу — це не просто назва і обкладинка: це структуровані відомості про жанри, рік випуску, акторський склад або творців, рейтинг, синопсис, наявні версії та формати. Самостійне формування таких даних вимагає редакційної команди і фінансово недоступне для більшості нових сервісів. Натомість поширеним підходом є інтеграція з відкритими API-провайдерами метаданих, що надають заздалегідь структуровані дані; це дозволяє новій платформі стартувати з повноцінним каталогом одразу після першого розгортання.

У межах цієї дипломної роботи виконавець як Full-Stack розробник реалізував підсистему каталогу контенту, агрегатора метаданих з зовнішніх джерел та потокового перегляду відео для гібридного сервісу Bazinga. Сервіс Bazinga поєднує у собі дві окремі частини: Bazinga Comics, що надає доступ до читання коміксів та манги, та BazingaTV, що дозволяє переглядати відеоконтент потоково. Підсистема, реалізована у роботі, охоплює увесь шлях відображення та взаємодії з контентом — від домашньої сторінки з трендовими рядами до повноекранного відеоплеєра.

Метою роботи є проектування, реалізація та інтеграція повноцінної підсистеми каталогу, агрегатора метаданих та потокового відео для сервісу Bazinga, що відповідає сучасним вимогам користувацького досвіду, продуктивності та можливості подальшого розвитку. Реалізація обіймає як серверну частину (ASP.NET Core, Entity Framework Core, MySQL, інтеграція з зовнішніми API), так і клієнтську (React, TypeScript, Tailwind CSS, HTML5 Video).

Завданнями дослідження є:

- аналіз доступних відкритих API-провайдерів метаданих для аніме/манги, коміксів і серіалів;
- проектування серверного агрегатора з підтримкою кешування результатів;
- проектування клієнтських сторінок каталогу з пагінацією, фільтрами і пошуком;
- реалізація центрального пошукового компонента, що працює за всіма джерелами одночасно;

- реалізація повноекранного відеоплеєра з підтримкою керування субтитрами, якістю та позицією;
 - проведення інтеграційного тестування на охопленому функціоналі.
-

ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ

Загальний опис

Програмне забезпечення (ПЗ) призначене для створення розподіленої інформаційної платформи Bazinga, що надає кінцевим користувачам можливість доступу до двох категорій контенту — коміксів/манги та потокового відео — за моделлю місячної підписки.

Архітектурно ПЗ ділиться на дві логічні половини. Перша половина — *Bazinga Comics* — забезпечує перегляд каталогу коміксів і манги, читання випусків онлайн (без можливості завантаження файлів локально), а також персоналізовану рекомендацію через систему фільтрів та інтеграцію з зовнішніми метаджерелами. Друга половина — *BazingaTV* — забезпечує потоковий перегляд відеоконтенту (мультсеріалів, аніме, ігрового відеоконтенту), агрегований через інтеграцію з відкритими метаджерелами на кшталт TVMaze.

Спільні елементи обох частин охоплюють:

- модель облікового запису з підтримкою до п'яти профілів на один обліковий запис (за зразком Netflix);
- модель підписки із трьома базовими тарифами — Bazinga Comics, BazingaTV та Bazinga Unlimited — і пробним семиденним тарифом, що автоматично переходить в один із оплачених при успішній прив'язці платіжного інструменту;
- адаптивний дизайн із підтримкою світлої і темної теми;
- універсальний пошук, що працює одночасно за коміксами, мангою та персонажами.

Призначення

ПЗ призначене для забезпечення зручної взаємодії між кінцевим користувачем (передплатником) та цифровим контентом, поданим у вигляді коміксів та потокового відео. Підсистема каталогу контенту та потокового відео, реалізована у межах цієї роботи, призначена для виконання таких користувацьких сценаріїв:

- перегляду каталогу контенту, поділеного на тематичні ряди (Top, Trending, New, Categories тощо);
- ефективного пошуку за назвою, серією, творцем або іменем персонажа;
- фільтрації каталогу за серією, творцем, жанром, рейтингом;
- ознайомлення з детальним описом конкретної одиниці контенту (синопсис, рейтинг, жанри, акторський склад/творці);
- запуску потокового відтворення відеоконтенту у вбудованому плеєрі та у повноекранному режимі;

- збереження одиниць контенту у списку бажаного (Wishlist) та персональний бібліотеці (Library);
- ознайомлення з останніми оновленнями платформи через стрічку новин;
- ознайомлення з юридичною та маркетинговою інформацією про сервіс.

Функціональні вимоги до підсистеми каталогу контенту, метаданих та потокового відео

Стартова (домашня) сторінка Bazinga Comics:

- карусель банерів з рекламою флагманських тайтлів і кнопкою СТА;
- блок фільтрів «Browse by» з двома вкладками: Series, Creator (без вкладки Character — за результатами користувацьких тестів);
- ряд «New This Week» — змішаний контент: 3 нові комікси і 3 топ-манги, перемішані за принципом інтерлівінгу;
- ряд «Comics» — 18 одиниць у вигляді 3-рядного блоку з кнопкою «See all»;
- ряд «Manga Universe» — 18 одиниць манги з кнопкою «See all»;
- спільна стилістика всіх рядів (MediaCard) з ефектами hover-підняття, червоного світіння та підпису «READ» при наведенні.

Каталог усіх коміксів (/comics/all):

- сітка коміксів з пагінацією Previous / Next;
- розмір сторінки: 18 одиниць (3 ряди по 6 на lg);
- підтримка фільтрації за ?search= (через центральний пошук);
- підтримка глибокого посилання ?openComic=<id> для відкриття картки у спливаючому вікні.

Каталог усієї манги (/manga):

- сітка манги з пагінацією Previous / Next;
- набір фільтрів-тегів за жанром (з дедуплікацією через перетин Genres + Themes у Jikan);
- розмір сторінки: 18 одиниць;
- підтримка ?q= для пошуку через MyAnimeList;
- підтримка ?openManga=<id> для глибокого посилання.

Каталог усіх персонажів (/characters):

- сітка карток персонажів з пагінацією;
- набір фільтрів за видавцем (Marvel, DC, Dark Horse, Image тощо), що отримуються динамічно з API;

- візуальна диференціація за моральним вирівнюванням (червона рамка для злодіїв, жовта для нейтральних, основна для героїв);
- підтримка ?q= та ?openHero=<id>.

Сторінка BazingaTV (/bazinga-tv):

- hero-карусель з автовідтворенням трейлерів локальних відеофайлів (з випадковим відрізком 10–15 секунд для кожного трейлера);
- ряд «Continue Watching»;
- ряд «Top 10 in Bazinga Today»;
- ряд «Trending Now», «Bazinga Originals», «Anime Universe», «New & Noteworthy»;
- ряд «Superhero Shows» з даними з TVMaze;
- підтримка фільтрації за жанром і пошуку через центральний компонент.

Сторінка перегляду відео (/bazinga-tv/watch/:id):

- повноекранний відеоплеєр HTML5;
- кнопка «Back» для повернення до BazingaTV;
- заголовок поточного контенту;
- обробка стану «контент не знайдено».

Центральний пошук (MasterSearch):

- відкривається через піктограму в шапці або комбінацію Cmd/Ctrl+K;
- візуально — повноекранний накладений шар з розмиттям підкладки;
- три категорії результатів: Comics (з локального каталогу), Characters (з akabab), Manga (з Jikan);
- кліком на результат відкривається відповідне модальне вікно безпосередньо у накладенні (без переходу на сторінку);
- кнопка «See all» в кожній категорії перенаправляє на відповідний каталог з підставленим пошуковим запитом.

Юридичні та інформаційні сторінки:

- сторінка «About Bazinga» з описом місії та статистики;
- сторінка «Help / FAQs» з пошуком і розгортувальними розділами питань-відповідей;
- сторінка «Careers» з переліком відкритих ролей;
- сторінка «Internships» з програмами стажування.

Нефункціональні вимоги

- час відповіді запитів до власних каталожних ендпоінтів не повинен перевищувати 200 мс при середньому навантаженні;
- запити до зовнішніх API кешуються у пам'яті терміном від 5 хвилин (списки) до 24 годин (списки жанрів);
- кеш статичного набору з ~731 персонажа akabab оновлюється не частіше рази на 12 годин;
- кеш результатів TVMaze має термін дії 6 годин;
- усі рядки каталогу повинні зберігати горизонтальну прокручуваність на сенсорних пристроях, на десктопі — додаткові кнопки прокручування;
- сторінки повинні автоматично прокручуватися на верх при кожній SPA-навігації (компонент `ScrollToTop`);
- адаптивний дизайн: підтримка екранів від 320 px до 4K;
- підтримуються браузер Chrome, Firefox, Safari, Edge у двох останніх стабільних версіях;
- усі картки контенту лінкуються виключно через React Router `<Link>`, без повної перезавантажки сторінки;
- відеоплеєр використовує нативні HTML5-можливості без зовнішніх бібліотек, що мінімізує розмір клієнтського бандла.

Архітектура

ПЗ спроектовано за клієнт-серверною моделлю з чітко відокремленими структурними вузлами:

- *Backend* — ASP.NET Core 9 Web API, що працює як stateless-сервіс і експонує REST-ендпоінти під префіксом `/api`;
- *Frontend* — SPA на React 18 + TypeScript, що збирається через Vite та обслуговується NGINX;
- *Database* — MySQL (через Pomelo.EntityFrameworkCore.MySql) у контейнері Docker;
- *External providers* — Jikan v4 (<https://api.jikan.moe/v4>), akabab/superhero-api (<https://akabab.github.io/superhero-api/api/>), TVMaze (<https://api.tvmaze.com>).

Технології

- Backend: C# 12, ASP.NET Core 9, Entity Framework Core 9, Pomelo MySQL 9.0, Microsoft.Extensions.Caching.Memory (вбудовано), System.Text.Json (вбудовано);

- Frontend: TypeScript 5.8, React 18.3, Vite 5, Tailwind CSS 3, Radix UI / shadcn/ui, TanStack Query 5, React Router 6, Lucide React (іконки), HTML5 Video API;
 - Інфраструктура: Docker, Docker Compose, NGINX.
-

РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ

У ході виконання дипломної роботи команда з двох Full-Stack розробників створила повнофункціональну веб-платформу Bazinga, що поєднує у собі сервіс читання коміксів і манги (*Bazinga Comics*) та сервіс потокового перегляду відеоконтенту (*BazingaTV*). Платформа призначена для роботи у моделі підписки і охоплює весь шлях користувача — від першого знайомства зі стартовою сторінкою до повноцінного перегляду контенту в рамках обраного тарифного плану.

Архітектурно система побудована за моделлю Single Page Application (SPA) з виокремленим бекендом, доступним через REST API. Цей підхід обрано як такий, що забезпечує найкращий компроміс між швидкістю відгуку інтерфейсу (за рахунок асинхронного клієнтського рендерингу без перезавантаження сторінки), гнучкістю масштабування (контейнеризований stateless-бекенд може горизонтально масштабуватися без обмежень) та зручністю розробки (чіткий поділ відповідальностей між клієнтом і сервером).

Серверна частина побудована на ASP.NET Core 9 з використанням Entity Framework Core поверх MySQL. Усі контролери побудовані відповідно до конвенцій REST: HTTP-методи GET / POST / PUT / DELETE використовуються згідно зі своїм семантичним значенням, відповіді мають стандартизовану форму JSON, а коди стану HTTP узгоджені з RFC 7231.

Клієнтська частина побудована на React 18 у поєднанні з TypeScript та зібрана за допомогою Vite. Для стилізації використовується утиліторна CSS-бібліотека Tailwind CSS, а для уніфікованих компонентів інтерфейсу — бібліотека shadcn/ui, побудована поверх примітивів Radix UI. Управління серверним станом виконується через TanStack Query.

Логічно платформа поділяється на дві великі підсистеми, що відповідають розподілу обов'язків між двома Full-Stack розробниками:

- 1. Підсистема облікових записів, профілів та підписок** (реалізована першим розробником) — охоплює реєстрацію через магічне посилання у пошті, автентифікацію за JWT, керування багатопрофільним обліковим записом, тарифи підписки, інтеграцію з платіжним шлюзом Stripe та сервісом надсилання електронної пошти MailKit.
- 2. Підсистема каталогу контенту, метаданих та потокового відео** (реалізована автором цього звіту) — охоплює відображення коміксів і манги, інтеграцію з відкритими метаджерелами (Jikan для аніме/манги, akabab/superhero-api для персонажів коміксів, TVMaze для серіалів), сторінку потокового перегляду, а також механізми пошуку та фільтрації.

Дві підсистеми взаємодіють між собою через спільний контекст автентифікації (`AuthContext` на стороні клієнта та `[Authorize]` атрибут на стороні сервера). Каталогна підсистема, реалізована у цій роботі, опирається на ідентифікацію поточного користувача та його тип підписки для: - визначення доступу до конкретного контенту; - розрахунку ціни одиниці контенту через сервіс `PricingService` (`Trial` і `Unlimited` — безкоштовно для цифрових копій; `Comics` — також безкоштовно; `TV` — не дає знижки на комікси; `Premium` — 25% знижки на цифрові копії, як `legacy`); - блокування доступу до контентних маршрутів через `guard`-компонент `RequireProfile`, що додатково перевіряє стан `Trial`-тарифу.

Розгортання здійснюється засобами `Docker` та `Docker Compose`: окремі контейнери для бекенду, клієнта (`NGINX`, що віддає статичні файли збірки `Vite`) та бази даних. Це забезпечує переносимість між середовищами розробки і виробничим середовищем без необхідності ручного налаштування.

Розробка велася з використанням системи контролю версій `Git` із застосуванням практик `pull request`, код-рев'ю та інтеграційного тестування на гілці перед злиттям у головну. У межах підсистеми каталогу окрему увагу приділено уніфікації компонентів: усі картки контенту (як комікси, так і манга) реалізовані через спільний компонент `MediaCard`, що забезпечує візуальну послідовність каталогу та усуває розбіжності у вигляді при додаванні нових типів контенту.

РОЗДІЛ 2. РЕАЛІЗАЦІЯ ПІДСИСТЕМИ КАТАЛОГУ ТА АГРЕГАТОРА МЕТАДАНИХ

2.1. Вибір зовнішніх API-провайдерів метаданих

Перед початком реалізації проведено аналіз доступних API-провайдерів метаданих для трьох ключових категорій контенту, передбачених у платформі Bazinga: коміксів супергеройської тематики, манги (японські комікси) та серіалів (для блоку Superhero Shows у BazingaTV).

Для манги та аніме обрано Jikan v4 (<https://api.jikan.moe/v4>). Підставою обрання стало: - відсутність вимоги до автентифікації — обмеження за обсягом запитів регулюється виключно публічними IP-rate-limit'ами (3 запити/сек, 60/хв); - покриття — API є проксі-сервером до бази MyAnimeList (~24 000 аніме та ~95 000 манги); - багатство метаданих — для кожної одиниці доступні назви (оригінальна, англійська, японська), синопсис, рейтинг, ранг, тип, кількість серій або глав, статус публікації, рік, акторський склад/творці, жанри, теми, посилання на трейлер; - безкоштовність і відсутність необхідності реєстрації облікового запису.

Для персонажів коміксів обрано akabab/superhero-api (<https://akabab.github.io/superhero-api/api/all.json>). Цей провайдер є статичним JSON-файлом, розміщеним на GitHub Pages, що містить ~731 персонажа з різних видавництв (Marvel, DC, Dark Horse, Image та інших). Підставою обрання стало: - відсутність будь-якої автентифікації та rate-limit'ів; - багатий набір метаданих: ім'я, повне ім'я, псевдоніми, видавець, моральне вирівнювання (good / bad / neutral), стать, раса, заняття, групова приналежність, місце народження, чотири розміри зображення (xs, sm, md, lg); - кількісні характеристики (powerstats) для кожного персонажа: інтелект, сила, швидкість, витривалість, сила, бойовий навичка — на шкалі від 0 до 100; - статичність даних — стабільні, не змінюються, що ідеально для довготривалого кешування.

Альтернативні джерела (офіційний Marvel API, ComicVine, SuperHero API на superheroapi.com) було відхилено через вимогу реєстрації, обмеження безкоштовного рівня (3000 запитів/добу для Marvel) або вимогу складного підписування кожного запиту (Marvel вимагає MD5-хеш ts + private + public). Безкоштовний Marvel Metadata API від emgroparker.com також було відхилено — він не містить URL обкладинок.

Для серіалів обрано TVMaze (<https://api.tvmaze.com>). Підставою обрання стало: - відсутність автентифікації; - щедрий ліміт запитів; - багатий набір метаданих: назва, тип, мова, жанри, статус, тривалість серії, дата прем'єри, синопсис у HTML-форматі, обкладинки medium/original, телемережа, рейтинг; - покриття як англомовного, так і регіонального контенту.

2.2. Серверний агрегатор метаданих

Серверний агрегатор реалізовано у вигляді двох спеціалізованих сервісів: `JikanService` для роботи з Jikan v4 та `SuperheroService` для роботи з akabab та TVMaze. Обидва сервіси зареєстровані як scored-залежності через `IHttpClientFactory` з налаштованим базовим URL та таймаутом 15 секунд.

JikanService експонує наступні методи:

- `TopAnimeAsync` — топ аніме (`/top/anime`);
- `SeasonNowAsync` — поточний сезон (`/seasons/now`);
- `SeasonUpcomingAsync` — майбутній сезон (`/seasons/upcoming`);
- `SearchAnimeAsync` — пошук з фільтрами `q`, `genre`, `status`, `type`, `orderBy`;
- `AnimeDetailAsync` — детальна інформація про конкретне аніме;
- `AnimeGenresAsync` — список жанрів аніме;
- аналогічний набір методів для манги (`TopMangaAsync`, `SearchMangaAsync`, `MangaDetailAsync`, `MangaGenresAsync`).

Для роботи з кешем використовується вбудована бібліотека `Microsoft.Extensions.Caching.Memory`. Конфігурація кешу:

- списки (топи, сезонні, пошукові результати) — 15 хвилин;
- деталі окремих одиниць — 60 хвилин;
- списки жанрів — 24 години.

Для зменшення кількості повторного коду використовується generic-метод `GetPagedCached<TRaw, TDto>` з явно вказаними параметрами типу при кожному виклику. Метод бере шлях запиту, TTL кешу та функцію відображення з сирого формату Jikan у власний DTO; повертає `JikanPagedResponse<TDto>` з даними та пагінацією.

Окрему увагу приділено мапінгу метаданих з формату Jikan у власний формат: жанри та теми об'єднуються у єдиний список через `.Concat(a.Themes ?? new())` (адже Jikan розділяє їх формально, але семантично користувач сприймає Action як жанр незалежно від того, чи це Genre чи Theme); URL трейлера витягується з вкладеного `trailer.embed_url`; URL зображення — з `images.jpg.large_image_url` з fallback на `images.jpg.image_url`.

SuperheroService експонує наступні методи:

- `ListAsync(page, limit, query, publisher, alignment)` — пагінований список з фільтрами;
- `GetByIdAsync(id)` — детальна інформація про конкретного персонажа;

- `GetPublishersAsync()` — список усіх видавців у датасеті (для генерації фільтрів);
- `GetCuratedShowsAsync()` — куратований список супергеройських серіалів;
- `SearchShowsAsync(query)` — пошук серіалів за назвою;
- `GetShowByIdAsync(id)` — детальна інформація про серіал.

Стратегія кешування для akabab: - весь датасет (~731 запис) завантажується один раз з `/all.json` (близько 1 МБ JSON) і кешується на 12 годин; - усі операції (фільтрація, пошук, пагінація) виконуються вже над кешованим списком у пам'яті — це усуває будь-яку залежність від доступності GitHub Pages під час нормальної роботи; - при першому зверненні до GitHub Pages запит має таймаут і обробку помилок (`try/catch ... return new List<SuperheroDto>()`), що забезпечує плавне зниження якості сервісу при тимчасовій недоступності зовнішнього джерела.

Стратегія для TVMaze: - для куратованого списку «супергеройських» серіалів використовується набір з 27 пошукових запитів (`marvel`, `dc comics`, `batman`, `superman`, `spider-man`, `x-men`, `avengers`, `flash`, `arrow`, ...) — оскільки TVMaze не має тегу «superhero», ми зводимо результати запитів за відомими брендами/іменами та об'єднуємо у єдиний список з дедуплікацією за `show.id`; - кеш термін дії 6 годин; - результати сортуються за наявністю обкладинки і датою прем'єри.

2.3. Серверні контролери

MetadataController є точкою входу для всіх запитів метаданих з клієнта. Експонує два набори ендпоінтів:

Для Jikan: - `GET /api/metadata/anime/top` - `GET /api/metadata/anime/season-now` - `GET /api/metadata/anime/season-upcoming` - `GET /api/metadata/anime/search` (фільтри `q`, `genre`, `status`, `type`, `orderBy`) - `GET /api/metadata/anime/{id}` - `GET /api/metadata/anime/genres` - (та аналогічні для манги)

Для супергеройських джерел: - `GET /api/metadata/superheroes` (`page`, `limit`, `q`, `publisher`, `alignment`) - `GET /api/metadata/superheroes/{id}` - `GET /api/metadata/superheroes/publishers` - `GET /api/metadata/superhero-shows` - `GET /api/metadata/superhero-shows/search` - `GET /api/metadata/superhero-shows/{id}`

Усі ендпоінти повертають уніфіковані DTO з camelCase JSON (для прямої сумісності з TypeScript-типами клієнта), не містять чутливих заголовків від оригінальних API і не пропускають оригінальні rate-limit-метрики.

ComicsController обслуговує власну базу коміксів — повноцінний CRUD з пошуком, фільтрацією та пагінацією. Підтримує сценарії як для роздрібного відображення (GET /api/comics), так і для редакційних операцій адміністратора.

CategoriesController надає список категорій (з підтримкою CRUD для адмінів через AdminComicsController).

CartController, WishlistController, LibraryController забезпечують класичні операції додавання, перегляду і видалення у кошику, списку бажаного та бібліотеці користувача. Усі захищені атрибутом [Authorize].

NewsController експонує редакційні новини, з фоновим сервісом NewsCleanupService, що періодично видаляє застарілі публікації.

AdminUsersController, AdminComicsController реалізують адміністративні операції (пошук користувачів, керування ролями, CRUD коміксів і категорій).

2.4. Сервіс ціноутворення

Сервіс PricingService визначає ціну одиниці контенту з урахуванням типу підписки користувача та формату покупки (цифрова чи фізична копія):

- **Trial і Unlimited** — цифрова копія безкоштовна, фізична зі знижкою 50%;
- **Comics** — спеціалізований тариф з безкоштовним цифровим читанням, фізична за повну ціну;
- **TV** — без жодних знижок на комікси (тариф націлений на відеоконтент);
- **Premium** (історичний) — 25% знижка на цифрову копію;
- **Free** або відсутність активної підписки — повна ціна.

Активність підписки визначається допоміжним методом IsSubscriptionActive, який перевіряє, чи SubscriptionType не є Free/null, а також чи SubscriptionExpiration не у минулому.

2.5. Клієнтська архітектура каталогу

Клієнтська частина побудована за принципом компонентної архітектури з чіткою ієрархією:

Уніфікований компонент MediaCard — спільна точка відображення для всіх типів контенту. Приймає чотири пропси: image, title, subtitle (під назвою), та опціональний score (рейтинг у верхньому правому куті). При наведенні: підняття картки на 8 px, масштабування обкладинки до 110%, радіальне червоне світіння, поява плашки «READ» внизу. Цей компонент покриває як комікси, так і мангу, забезпечуючи візуальну єдність каталогу.

Компонент ComicCard — тонка обгортка над **MediaCard**, що зберігає сумісність зі старим API (де параметр називався `creators`, а не `subtitle`).

Компонент ComicSection — generic-компонент ряду карток з заголовком і кнопкою «See all». Приймає масив `items` типу `T` `extends ComicLike` та функцію `onComicClick(item: T)`, що дає змогу зберегти дискримінатор типу при роботі зі змішаним рядом (на домашній сторінці ряд «New This Week» містить як комікси, так і мангу).

Компонент MangaUniverse — складний компонент-секція з двома режимами: - `mode="home"` — короткий 18-картковий ряд для домашньої сторінки з кнопкою «See all»; - `mode="full"` (за замовчуванням) — повноцінний каталог з фільтрами жанрів, пагінацією і станом завантаження.

Завантаження жанрів виконується через `useQuery({ enabled: !isHome })` — тобто на домашній сторінці зайвий запит не робиться. Список жанрів дедуплікується за `lowercase`-назвою, оскільки Jikan повертає перетин `Genres + Themes` (Action може з'являтися двічі).

Компонент SuperheroesSection реалізує каталог персонажів з: підбіркою чипів-видавців, що завантажуються з `GET /api/metadata/superheroes/publishers`; пагінованою 6-колонковою сіткою; візуальною диференціацією за моральним вирівнюванням (рамка картки: червона для bad, жовта для neutral, червона-основна для good); підтримкою глибокого посилання `?openHero=<id>` через ефект, що автоматично відкриває модальне вікно деталей.

Компонент SuperheroShowsSection реалізує ряд супергеройських серіалів на сторінці BazingaTV з власним модальним вікном для деталей TVMaze.

2.6. Сторінки каталогу

Index.tsx (домашня сторінка Bazinga Comics) поєднує hero-карусель з обкладинками тайтлів, блок фільтрів «Browse by» (Series, Creator), ряд «New This Week» зі змішаними коміксами та мангою, ряд «Comics» з 18 картками та посиланням на `/comics/all`, ряд **MangaUniverse** у режимі `home` з посиланням на `/manga`. Кожен ряд карток обробляє кліки через спільний обробник `handleTileClick`, що різнить тип контенту по дискримінатору `_comic` або `_manga` і відкриває відповідне модальне вікно.

AllComics.tsx — каталог усіх коміксів з пагінованою сіткою 18 одиниць на сторінку. Підтримує параметр `?search=<query>` для пошуку та `?openComic=<id>` для глибокого посилання.

AllManga.tsx — тонка обгортка, що використовує **MangaUniverse** у режимі `full`.

Characters.tsx — тонка обгортка над **SuperheroesSection**.

2.7. Модальні вікна деталей

Реалізовано три модальних компонента, що використовуються як з каталожних сторінок, так і з центрального пошуку:

ComicModal — деталі коміксу: обкладинка, назва, ПІБ автора, опис, кнопка «Read Now», список тегів. Працює з даними з власної БД або з placeholder-даних.

MangaModal — деталі манги: обкладинка, оригінальна та англійська назви, рейтинг, ранг, тип, кількість глав/томів, статус публікації, синопсис, автори, жанри, кнопка «Read Now», посилання на MyAnimeList. При відкритті автоматично запитує повну детальну інформацію через `GET /api/metadata/manga/{id}` (із кешуванням через TanStack Query).

HeroModal — деталі персонажа: велике зображення, ім'я, повне ім'я, видавець, моральне вирівнювання, раса, псевдоніми, графічні смужки кількісних характеристик (powerstats) з градієнтним заповненням від основного до помаранчевого, заняття, групова приналежність, місце народження.

Усі три модальних компонента стилізовані за спільним мовленням: розмитий тло-план, рамка з кольором акценту, тінь у вигляді радіального червоного світіння, кнопка закриття у верхньому правому куті.

РОЗДІЛ 3. РЕАЛІЗАЦІЯ ПІДСИСТЕМИ ПОТОКОВОГО ВІДЕО ТА ПОШУКУ

3.1. Сторінка BazingaTV

Сторінка BazingaTV (/bazinga-tv) є основним інтерфейсом перегляду відеоконтенту у платформі. Її структура повторює загальновідому модель Netflix-каталогу: hero-карусель з автовідтворенням трейлерів, далі — серія тематичних рядів з прокручуванням списком карток.

Hero-карусель (TVHero) реалізована як автоматично перемикальний шар з кількома трейлерами, що містяться у каталозі `public/trailers/`. Поведінка карусели:

- кожні 10–15 секунд відбувається перехід до наступного трейлера (точний інтервал обирається випадково для кожного перегляду, щоб уникнути механічного враження);
- при кожному показі трейлера обирається випадкова стартова позиція у межах першої третини відео — це усуває візуальну монотонність повторного перегляду;
- доступне ручне керування через стрілки вліво/вправо та через крапки-індикатори знизу;
- паралельно з відеоряду відображається опис поточного трейлера (назва, тип, рік, синопсис) у лівій частині;
- розмір і позиція карусели прив'язані до висоти екрана (80vh з мінімумом 600px), що забезпечує кінотеатральне враження.

Рядні компоненти ділять каталог на тематичні зрізи: - «Continue Watching» — список з прогрес-індикаторами, заглушка для майбутньої функції відстеження прогресу; - «Top 10 in Bazinga Today» — топ-10 аніме з гранично-великими цифрами рангу; - «Trending Now» — поточний сезон з Jikan; - «Bazinga Originals» — добірка популярних з фільтром `orderBy=popularity`; - «Anime Universe» — фільтрований за обраним жанром або просто за рейтингом; - «New & Noteworthy» — майбутній сезон; - «Superhero Shows» — серіали супергеройської тематики з TVMaze.

Кліком на картку у будь-якому ряду відкривається модальне вікно `ShowModal` з повним описом, рейтингом, кнопками «Play», «My List», «Like», «Not for me».

3.2. Сторінка повноекранного перегляду

Сторінка `/bazinga-tv/watch/:id` реалізує повноекранний відеоплеєр HTML5. На вхід вона приймає ідентифікатор трейлера, який зіставляється зі статичним списком у `data/trailers.ts`. Якщо трейлер не знайдено — відображається повідомлення «Trailer not found» з кнопкою повернення до BazingaTV.

Плеєр використовує нативні елементи `<video>` з атрибутами `controls`, `autoPlay`, `playsInline`. Це забезпечує: - підтримку всіх стандартних елементів керування (відтворення/пауза, прокручування, гучність, повноекранний режим); - сумісність з усіма сучасними браузерами без зовнішніх бібліотек; - мінімальний розмір клієнтського бандла (відсутність third-party відеоплеєрів типу `video.js`, `plyr`, `Vidstack`).

Поверх відео виводиться кнопка «Back» (з іконкою стрілки вліво) для повернення до BazingaTV. Назва поточного контенту відображається у верхньому правому куті в напівпрозорому стилі.

3.3. Центральний пошук

Центральний пошук (`MasterSearch`) реалізований як накладений шар, що відкривається поверх будь-якої сторінки. Активація: - кліком на іконку лупи у шапці; - комбінацією клавіш `Cmd+K` (macOS) або `Ctrl+K` (Windows/Linux).

Перші версії компонента мали два проблематичних аспекти: 1. Підказки-приклади (`Try: Berserk, Spider-Man, Night Cipher`) використовували нативні DOM-подію `input.dispatchEvent(new Event("input"))` для заповнення поля вводу. У React контрольований інпут на нативні DOM-події не `reacts` (state setter не викликається), і це призводило до того, що клік по підказці фактично нічого не робив. виправлення — пряме виклик `setQuery(promptValue)`. 2. Шар був реалізований як центрована модалка обмеженого розміру, що порушувало користувацьке очікування «накладеного панорамного шару». виправлення — повноекранний шар з розмитим фоном (як у першій версії).

Поточна реалізація: - повноекранний шар з `fixed inset-0 z-[90] bg-black/85 backdrop-blur-md`; - мітка «Search Bazinga» у заголовку; - велике поле вводу з фокусом за замовчуванням, кнопкою очищення (X) та ладером під час асинхронного запиту; - три підказки-приклади як активовані кнопки; - результати поділені на три категорії: `Comics` (з локального плейсхолдер-каталогу), `Characters` (через `fetchSuperheroes`), `Manga` (через `searchManga`); - кожна категорія обмежена 5 результатами для уникнення необхідності прокручування; - кліком на результат відкривається відповідне модальне вікно (`ComicModal`, `HeroModal`, `MangaModal`) безпосередньо у накладенні — без переходу на каталог; - кнопка «See all» у заголовку

категорії переходить на відповідний каталог з підставленим пошуковим запитом; - послідовний `Escape`: спершу закриває модальне вікно (якщо відкрите), потім — шар пошуку.

Дебаунс введення складає 250 мс — це знаходить компроміс між миттєвим відгуком та не-надмірним навантаженням на API. Вихідні запити блокуються до досягнення мінімум двох символів.

3.4. Стрічка новин та юридичні сторінки

Сторінка `/news` відображає стрічку редакційних публікацій, отриманих з `GET /api/news`. Кожна публікація має заголовок, автора, дату, текст у форматі Markdown. Авторизовані користувачі з адміністративною роллю можуть створювати нові публікації безпосередньо зі сторінки. Фоновий сервіс `NewsCleanupService` періодично видаляє публікації, що прострочили свій `ExpiresAt`.

Юридичні та маркетингові сторінки реалізовані як статичні React-компоненти з адаптивним стилем і анімаціями `hover`:

- **AboutBazinga** — місія компанії, 4 ключові принципи (читання, потокове, креатори, без реклами), цифри-статистика (42М читань на місяць, 2300+ серій, 180 країн, \$14М роялті), історія Bazinga;
- **Faqs** — пошуковий індекс по чотирьох категоріях (Account, Reading, BazingaTV, Profiles) з розгортувальними блоками питань і кнопками контакту через e-mail або open ticket;
- **Careers** — опис чотирьох команд (Engineering, Design, Editorial, BazingaTV), список з 6 відкритих вакансій з тегами міст і типу зайнятості, перелік 3 пільг;
- **Internships** — 4 програми стажування з тривалістю, місцем та описом, заклик до подачі заявок.

Усі сторінки використовують hero-секцію з радіальним градієнтним підсвічуванням, єдиний шрифт-стиль і анімовані картки з підняттям при наведенні.

3.5. Універсальна навігація

Реалізовано загальний компонент `Header` з підрядковим меню (News, Comics, Characters, Manga, Library, TV), іконкою пошуку (запускає `MasterSearch`), кнопкою «Join Now» (для не-підписаних користувачів) та користувацьким меню. Меню адаптується для мобільних пристроїв через `Sheet` з Radix UI.

Загальний компонент `Footer` містить логотип, посилання на чотири розділи (About, Help, Careers, Internships), іконки соціальних мереж (Facebook, Twitter, Instagram, YouTube, Twitch), копірайтний рядок та юридичні посилання.

Окремий компонент `ScrollToTop` (`useEffect` на зміну `pathname` з виклику `window.scrollTo(0, 0)`) гарантує, що при кожному SPA-переході вікно повертається на верх — типове користувацьке очікування, яке React Router не реалізує за замовчуванням.

3.6. Адміністративна частина

Адміністративна сторінка (`/admin`) доступна лише користувачам з роллю `ADMIN`. Вона надає інтерфейс для:

- управління користувачами: пошук, перегляд, зміна ролі, блокування, видалення;
- управління коміксами: створення, редагування, видалення, керування зображеннями;
- управління категоріями і умовами стану;
- модерації відгуків (планується);
- перегляду статистики платформи (планується).

Усі адміністративні операції захищені на серверній стороні через перевірку JWT-клейму `role == "ADMIN"`.

ВИСНОВКИ

У ході виконання дипломної роботи спроектовано та повністю реалізовано підсистему каталогу контенту, агрегатора метаданих та потокового відео для гібридної інформаційної платформи Bazinga. Реалізація охоплює як серверну частину (ASP.NET Core 9 + Entity Framework Core + MySQL з інтеграцією трьох зовнішніх API-провайдерів), так і клієнтську (React 18 + TypeScript + Vite + Tailwind CSS).

Серверна частина забезпечує: - проксіювання і кешування запитів до Jikan v4 (~24 000 аніме та ~95 000 манги, з кешем 15 хв – 24 год); - проксіювання і кешування запитів до akabab/superhero-api (~731 персонажа, з кешем 12 год); - проксіювання і кешування запитів до TVMaze (куратований список з 27 пошукових запитів супергеройської тематики, з кешем 6 год); - повноцінний CRUD над власним каталогом коміксів, категоріями та умовами; - персональні зрізи (Cart, Wishlist, Library); - стрічку новин з фоновим прибиральником застарілих публікацій; - сервіс ціноутворення з урахуванням типу підписки користувача.

Клієнтська частина реалізує: - домашню сторінку Bazinga Comics з hero-каруселлю, фільтрами Browse by, рядом New This Week (зі змішаним контентом), рядом Comics і рядом Manga Universe; - сторінку каталогу всіх коміксів з пагінацією; - сторінку каталогу всієї манги з фільтрами жанрів і пагінацією; - сторінку каталогу всіх персонажів з фільтрами видавців та візуальною диференціацією за моральним вирівнюванням; - сторінку BazingaTV з hero-каруселлю трейлерів і шістьма тематичними рядами аніме плюс рядом супергеройських серіалів; - сторінку повноекранного відтворення відео; - центральний пошук, що працює одночасно за коміксами, мангою і персонажами з відкриттям модальних вікон деталей безпосередньо у накладенні; - три уніфікованих модальних вікна деталей (ComicModal, MangaModal, HeroModal); - стрічку новин і чотири юридичних/маркетингових сторінки (About, FAQs, Careers, Internships).

У ході реалізації виконано такі ключові завдання:

- проаналізовано та обрано три безкоштовних API-провайдери метаданих (Jikan, akabab, TVMaze), що покривають усі категорії контенту платформи без вимоги до автентифікації;
- спроектовано та реалізовано серверну архітектуру кешування, що мінімізує кількість зовнішніх запитів і забезпечує плавне зниження якості сервісу при тимчасовій недоступності зовнішніх джерел;
- реалізовано generic-метод `GetPagedCached<TRaw, TDto>` для усунення повторного коду у `JikanService`;

- спроектовано та реалізовано уніфікований компонент `MediaCard`, що використовується як для коміксів, так і для манги, забезпечуючи візуальну послідовність каталогу;
- реалізовано generic-компонент `ComicSection`, що дозволяє передавати дискримінатор типу контенту через ряд карток і коректно маршрутизувати кліки до відповідних модальних вікон;
- реалізовано допоміжний компонент `MangaUniverse` з двома режимами (home preview / full catalog) та коректним дедуплікуванням списку жанрів;
- реалізовано центральний пошуковий компонент `MasterSearch` з повноекранним накладенням, дебаунсом введення, трьома категоріями результатів та локальним відкриттям модальних вікон;
- реалізовано анімовану hero-карусель `TVHero` з автоматичним перемиканням трейлерів кожні 10–15 секунд та випадковою стартовою позицією;
- забезпечено підтримку глибоких посилань (`?search=`, `?q=`, `?openComic=<id>`, `?openManga=<id>`, `?openHero=<id>`) на всіх каталожних сторінках;
- реалізовано допоміжний компонент `ScrollToTop` для коректної SPA-навігації;
- здійснено інтеграційне тестування у браузерях Chrome, Firefox, Edge для перебігу повних сценаріїв перегляду каталогу, пошуку, відкриття деталей та відтворення відео.

Особливу увагу приділено уніфікації компонентного словника. До початку фінального етапу реалізації картки коміксів і манги мали різний вигляд (різні стилі заголовку, різні поведінки наведення, різна відсутність/наявність плашки рейтингу). Після виділення спільного `MediaCard` усі типи карток стали ідентичними з точки зору поведінки, а додавання нової категорії контенту (наприклад, аудіокниг) у майбутньому буде потребувати лише розширення `MediaCard` опціональним параметром.

Окремою архітектурною знахідкою стала стратегія fallback-каталогу: якщо власна база коміксів пуста (наприклад, при першому розгортанні), клієнт автоматично відображає 12 куратованих placeholder-коміксів з пов'язаним списком серій, персонажів і творців, що дозволяє користувачу одразу побачити повноцінну домашню сторінку без необхідності попереднього наповнення БД. При появі реальних записів у БД placeholder-каталог автоматично замінюється на справжній.

Обрані технології та архітектурні рішення повністю задовольнили вимоги технічного завдання. Підсистема є завершеною, працездатною, відмовостійкою та готовою до подальшого розвитку — як з боку додаткових функцій (наприклад, реалізації функцій

«Continue Watching» з реальним відстеженням прогресу, інтеграції з додатковими метаджерелами на кшталт TMDb для серіалів, додавання повноцінного HLS-стрімінгу для відеоконтенту), так і з боку масштабування.

Подальшими напрямками розвитку можуть бути: інтеграція з Stripe Connect для виплат авторам коміксів; впровадження повноцінного DRM для захисту авторських прав на стрімінговий контент; реалізація рекомендаційної системи на основі історії перегляду та оцінок; додавання офлайн-режиму через Service Worker для попереднього завантаження окремих випусків.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

- ASP.NET Core Documentation — <https://learn.microsoft.com/aspnet/core>
- Entity Framework Core Documentation — <https://learn.microsoft.com/ef/core>
- Pomelo.EntityFrameworkCore.MySql — <https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql>
- Microsoft.Extensions.Caching.Memory — <https://learn.microsoft.com/aspnet/core/performance/caching/memory>
- React Documentation — <https://react.dev/learn>
- TypeScript Documentation — <https://www.typescriptlang.org/docs/>
- Vite Documentation — <https://vite.dev/guide/>
- Tailwind CSS Documentation — <https://tailwindcss.com/docs>
- shadcn/ui — <https://ui.shadcn.com/>
- Radix UI Primitives — <https://www.radix-ui.com/primitives>
- TanStack Query — <https://tanstack.com/query/latest>
- React Router 6 — <https://reactrouter.com/>
- Lucide React (icon set) — <https://lucide.dev/>
- Jikan v4 — Unofficial MyAnimeList API — <https://docs.api.jikan.moe/>
- akabab/superhero-api — <https://akabab.github.io/superhero-api/api/>
- TVMaze API — <https://www.tvmaze.com/api>
- HTML5 Video API — <https://developer.mozilla.org/docs/Web/HTML/Element/video>
- MDN Fetch API — https://developer.mozilla.org/docs/Web/API/Fetch_API
- WCAG 2.1 (доступність) — <https://www.w3.org/TR/WCAG21/>
- Bearer Token RFC 6750 — <https://datatracker.ietf.org/doc/html/rfc6750>
- Netflix Browse UX Pattern — <https://help.netflix.com/en/node/41049>
- Репозиторій проєкту — <https://github.com/dentuss/Bazinga>